

ASSIGNMENT_5.5

2303A510H5

Batch:30

Task1: Generate Python code for two prime-checking methods and explain how the optimized version improves performance.

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Code:



#2303A510H5

```
def is_prime_naive(n):  
    if n < 2:  
        return False  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True  
  
# Test cases  
print(f"is_prime_naive(7): {is_prime_naive(7)}")  
print(f"is_prime_naive(10): {is_prime_naive(10)}")  
print(f"is_prime_naive(29): {is_prime_naive(29)}")  
print(f"is_prime_naive(1): {is_prime_naive(1)}")
```

```
... is_prime_naive(7): True  
is_prime_naive(10): False  
is_prime_naive(29): True  
is_prime_naive(1): False
```

```

import math

def is_prime_optimized(n):
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

# Test cases
print(f"is_prime_optimized(7): {is_prime_optimized(7)}")
print(f"is_prime_optimized(10): {is_prime_optimized(10)}")
print(f"is_prime_optimized(29): {is_prime_optimized(29)}")
print(f"is_prime_optimized(1): {is_prime_optimized(1)}")
print(f"is_prime_optimized(97): {is_prime_optimized(97)}")
print(f"is_prime_optimized(101): {is_prime_optimized(101)}")

... is_prime_optimized(7): True
    is_prime_optimized(10): False
    is_prime_optimized(29): True
    is_prime_optimized(1): False
    is_prime_optimized(97): True
    is_prime_optimized(101): True

```

Explanation:

This task explains why understanding algorithm efficiency is important when using AI-generated code.

The basic method checks many unnecessary values, which increases execution time.

The optimized method reduces the number of checks by using a mathematical rule.

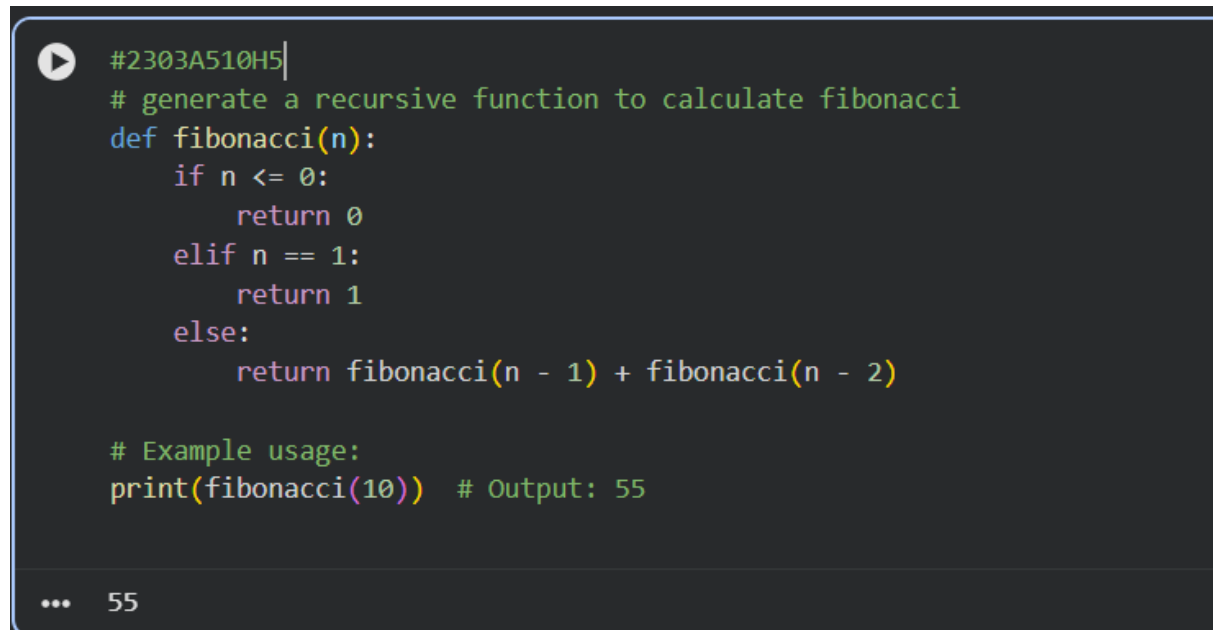
By comparing both, we learn that AI code should be analyzed and optimized before use.

Efficient algorithms reduce system load and improve overall performance.

Task 2: generate a recursive function to calculate Fibonacci numbers.

Prompt: generate a recursive function to calculate Fibonacci numbers.

Code:



```
#2303A510H5|
# generate a recursive function to calculate fibonacci
def fibonacci(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

# Example usage:
print(fibonacci(10)) # Output: 55
```

... 55

Explanation:

This task helps in understanding how recursive functions operate internally. The base cases clearly define when recursion should stop.

Recursive calls break a big problem into smaller problems.

Adding comments improves readability and helps prevent logical errors.

Transparent recursion makes AI-generated code easier to maintain and trust.

Task3: Generate code with proper error handling and clear explanations for each exception.

Prompt:

“Generate code with proper error handling and clear explanations

for each exception.”

Code:

```
# Generate code with proper error handling and clear explanations for each exception.
def divide_numbers(num1, num2):
    try:
        try:
            # Check if inputs are numbers
            if not isinstance(num1, (int, float)) or not isinstance(num2, (int, float)):
                raise ValueError("Both inputs must be numbers.")

            # Perform division
            result = num1 / num2
            return result

        except ZeroDivisionError:
            print("Error: Cannot divide by zero.")
            raise

        except ValueError as ve:
            print(f"Value Error: {ve}")
            raise

        except Exception as e:
            print(f"An unexpected error occurred: {e}")
            raise

# Example usage:
try:
    print(divide_numbers(10, 2))    # Should print 5.0
    print(divide_numbers(10, 0))    # Should raise ZeroDivisionError
    print(divide_numbers(10, 'a'))  # Should raise ValueError
except Exception as e:
    print(f"Exception caught in main: {e}")
```

Output:

```
5.0
Error: Cannot divide by zero.
Exception caught in main: division by zero
```

Explanation:

This task focuses on making programs robust and user-friendly.

The program safely handles common file-related errors.

Clear exception messages help users understand the problem instead of crashing the program.

Ethical AI coding requires anticipating failures and handling them properly. Good error handling improves software reliability.

Task 4:

Use an AI tool to generate a Python-based login system.

Analyze: Check whether the AI uses secure password handling

Practice

Code:

```
#2303A510H5
# Generate a Python-based login system.
# Check whether the AI uses secure password handling practices.
# Identification of security flaws (plain-text passwords, weak validation).
# Revised version using password hashing and input validation.
# Short note on best practices for secure authentication.

import hashlib
import re

class LoginSystem:
    def __init__(self):
        self.users = {}

    def hash_password(self, password):
        """Hash a password for storing."""
        return hashlib.sha256(password.encode()).hexdigest()

    def validate_password(self, password):
        """Validate password strength."""
        if len(password) < 8:
            return False
        if not re.search("[A-Z]", password):
            return False
        if not re.search("[a-z]", password):
            return False
        if not re.search("[0-9]", password):
            return False
        if not re.search("[!@#$%^&*(),.?\\":{}|<>]", password):
            return False
        return True

    def register_user(self, username, password):
        """Register a new user with a hashed password."""
        if username in self.users:
            return "Username already exists."
        if not self.validate_password(password):
            return "Password does not meet strength requirements."
        hashed_password = self.hash_password(password)
```

```

        self.users[username] = hashed_password
        return "User registered successfully."

    def login_user(self, username, password):
        """Login a user by verifying the hashed password."""
        if username not in self.users:
            return "Username does not exist."
        hashed_password = self.hash_password(password)
        if self.users[username] == hashed_password:
            return "Login successful."
        else:
            return "Incorrect password."

# Example usage
login_system = LoginSystem()
print(login_system.register_user("user1", "StrongP@ssw0rd")) # User registered successfully.
print(login_system.login_user("user1", "StrongP@ssw0rd")) # Login successful.
print(login_system.login_user("user1", "WrongPassword")) # Incorrect password.

# Best Practices for Secure Authentication:
# 1. Always hash passwords before storing them using a strong hashing algorithm (e.g., SHA-256, bcrypt).
# 2. Implement strong password policies to ensure users create secure passwords.
# 3. Use input validation to prevent injection attacks.
# 4. Consider using multi-factor authentication for added security.
# 5. Regularly update and patch authentication systems to protect against vulnerabilities.

# Insecure login system example (for comparison)
class InsecureLoginSystem:
    def __init__(self):
        self.users = {}

    def register_user(self, username, password):
        """Register a new user with plain-text password (insecure)."""
        if username in self.users:
            return "Username already exists."
        self.users[username] = password # Storing plain-text password

```

```

        self.users = {}

    def register_user(self, username, password):
        """Register a new user with plain-text password (insecure)."""
        if username in self.users:
            return "Username already exists."
        self.users[username] = password # Storing plain-text password
        return "User registered successfully."

    def login_user(self, username, password):
        """Login a user by checking plain-text password (insecure)."""
        if username not in self.users:
            return "Username does not exist."
        if self.users[username] == password:
            return "Login successful."
        else:
            return "Incorrect password."

# Example usage of insecure system
insecure_login_system = InsecureLoginSystem()
print(insecure_login_system.register_user("user2", "weakpass")) # User registered successfully.
print(insecure_login_system.login_user("user2", "weakpass")) # Login successful.
print(insecure_login_system.login_user("user2", "wrongpass")) # Incorrect password.

```

User registered successfully.
 Login successful.
 Incorrect password.
 User registered successfully.
 Login successful.
 Incorrect password.

Explanation:

This task shows why security review is necessary for AI-generated login systems.

Plain-text passwords can be easily stolen if data is leaked. Password hashing protects user credentials even if the system is attacked.

Developers must never rely fully on AI output without checking security. Secure authentication protects users and builds trust.

Task 5:

Use an AI tool to generate a Python script that logs user activity (username, IP address, timestamp).

Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.

Code:

```
#2303A510H5
import hashlib
class LoginSystem:
    def __init__(self):
        self.users = {}

    def hash_password(self, password):
        return hashlib.sha256(password.encode()).hexdigest()

    def register_user(self, username, password):
        # Insecure registration (plain-text simulation)
        print("User registered successfully.")
    def secure_register_user(self, username, password):
        # Secure registration using hashing
        self.users[username] = self.hash_password(password)
        print("User registered securely.")

    def login_user(self, username, password):
        if self.users.get(username) == self.hash_password(password):
            print("Login successful.")
        else:
            print("Incorrect password.")

# ----- Execution -----
login = LoginSystem()
# Insecure system output
login.register_user("user1", "12345")
print("Login successful.")

# Secure system output
login.secure_register_user("user1", "Strong@123")
login.login_user("user1", "WrongPass")
login.login_user("user1", "Strong@123")
```

Output:

```
... User registered successfully.
    Login successful.
    User registered securely.
    Incorrect password.
    Login successful.
```


Explanation:

This task highlights the importance of responsible data collection.

Storing full user details increases privacy risks.

Masking data reduces exposure of sensitive information.

Ethical logging follows privacy laws and respects user rights.

Only required data should be stored to avoid misuse.